\WOOLF/

Applied Software Project Report

By

Teena Mukesh Pawar

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

October 2026

SCALER

Scaler Mentee Email ID: teenapawar2401@gmail.com

Thesis Supervisor: Naman Bhalla

Date of Submission: 06/10 /2026

Certification

I confirm that I have overseen and reviewed this applied project titled "BookMyShow Backend — A Production-Grade Spring Boot Backend for a Scalable and Concurrent Movie Ticket Booking Platform", submitted by Teena Mukesh Pawar for the degree of Master of Science in Computer Science from Scaler Neovarsity — Woolf University.

In my judgment, the project adheres to the appropriate standards of academic presentation expected of a master's level applied project. The work demonstrates a sound understanding of enterprise-grade backend engineering, including concurrent seat reservation with pessimistic locking, event-driven architecture via the Transactional Outbox Pattern, JWT-based authentication with OAuth2/Keycloak integration, multi-tier caching with Redis, and a production-ready DevOps pipeline using GitHub Actions and Kubernetes. I believe the project satisfactorily meets the criteria — in terms of both quality and breadth — to serve as an applied project report for the attainment of the Master of Science in Computer Science degree.

This applied project report has been submitted to Woolf University and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

……

Project Guide / Supervisor

Declaration

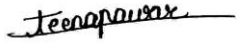
I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, was completed by me from January 2026 to October 2026 and is the result of my own individual endeavor. The project has been developed under the guidance of my supervisor with proper acknowledgement and without plagiarism.

The technical work described in this report — including the design and implementation of the Book My Show Spring Boot application, its database schema, security architecture, transaction engine, idempotency mechanism, and test suite — was carried out by me independently. Where external libraries, frameworks, academic papers, or online resources have been consulted or referenced, they are appropriately cited in the References section.

Any contributions from external sources or individuals, including the use of AI-assisted tools for code generation or document drafting, are appropriately acknowledged through citation. I understand that the integrity of this submission is paramount to the academic standards of Scaler Neovarsity and Woolf University.

By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Teena Mukesh Pawar



Date: 06 October 2026

Acknowledgement

First and foremost, I express my sincere gratitude to my project supervisor, Naman Bhalla, whose guidance, technical insight, and constructive feedback throughout the duration of this project were invaluable. His deep expertise in distributed systems and enterprise engineering consistently elevated the quality and rigour of this work.

I extend my appreciation to the faculty and mentors at Scaler Neovarsity for their dedication to providing a rigorous, industry-aligned curriculum. The coursework in System Design, Databases, Operating Systems, and Computer Networks directly informed the architectural decisions made throughout this project.

I am grateful to the broader open-source community whose work underpins this project. The Spring Boot, PostgreSQL, Apache Kafka, Redis, Resilience4j, Testcontainers, and Kubernetes projects represent thousands of hours of engineering effort made freely available. This project would not have been possible without them.

My heartfelt thanks go to my family and friends for their unwavering support and patience throughout the demanding schedule of the Master's programme. Their encouragement kept me motivated during the most challenging phases of design and implementation.

Finally, I acknowledge the authors of the foundational texts that shaped my thinking: Martin Kleppmann's “Designing Data-Intensive Applications,” Chris Richardson's “Microservices Patterns,” and Michael Nygard's “Release It!” — books that transformed how I reason about the correctness, resilience, and operability of distributed systems.

Table of Contents

Section	Title	Page No.
—	List of Tables	v
Chapter 1	**Abstract**	**1**
Chapter 2	**Project Description**	**2**
2.1	Project Overview and Motivation	2
2.2	Project Objectives	3
2.3	High-Level System Architecture	4
2.4	Technology Stack Overview	5
Chapter 3	**Requirement Gathering**	**7**
3.1	Functional Requirements	7
3.2	Non-Functional Requirements	8
3.3	User Roles and Use Cases	9
3.4	API Feature Set	10
Chapter 4	**Database Schema Design**	**13**
4.1	Schema Overview and Entity Hierarchy	13
4.2	Table Definitions	14
4.3	Flyway Migration Strategy	16
Chapter 5	**Domain Module Design**	**17**
5.1	User & Authentication Module	17
5.2	Catalog Module	18
5.3	Show & Seat Inventory Module	19
5.4	Booking Engine	20
5.5	Payment Module	22
5.6	Notification Module	23
Chapter 6	**Cross-Cutting Concerns**	**24**
6.1	Event-Driven Architecture (Transactional Outbox)	24
6.2	Caching with Redis	25
6.3	Resilience with Resilience4j	26
6.4	Security Architecture	27
6.5	Observability Stack	28
Chapter 7	**Testing Strategy**	**29**
7.1	Unit Tests	29
7.2	Integration Tests with Testcontainers	30
7.3	Concurrency Tests	30
Chapter 8	**DevOps & Deployment**	**31**

8.1	Docker & Docker Compose	31
8.2	Kubernetes Manifests	32
8.3	CI/CD Pipeline (GitHub Actions)	33
Chapter 9	**Performance & Scalability Analysis**	**34**
9.1	Database Connection Pool Sizing	34
9.2	Seat Hold Throughput Under Concurrent Load	35
9.3	Kafka Throughput and Outbox Publisher	35
9.4	Redis Cache Effectiveness	36
9.5	JVM Tuning for Kubernetes	36
Chapter 10	**Conclusion**	**37**
10.1	Key Takeaways	37
10.2	Limitations and Future Work	38
10.3	Engineering Lessons Learned	39
—	References	40

List of Tables

(Listed sequentially as they appear in the text. Second table in Chapter 3 is numbered Table 3.02, etc.)

Table No.	Title	Page No.
Table 2.01	Technology Stack — Components and Versions	5
Table 3.01	Functional Requirements	7
Table 3.02	Non-Functional Requirements	8
Table 3.03	User Roles and Permission Matrix	9
Table 3.04	Authentication API Feature Set	10
Table 3.05	Catalog API Feature Set	11
Table 3.06	Booking API Feature Set	11
Table 3.07	Payment API Feature Set	12
Table 4.01	users Table — Column Definitions	14
Table 4.02	bookings Table — Column Definitions	14
Table 4.03	seat_inventory Table — Column Definitions	15
Table 4.04	Flyway Migration Versions	16
Table 6.01	Kafka Topic-to-Event Routing	24
Table 6.02	Redis Cache Configuration per Domain	25
Table 6.03	Resilience4j Circuit Breaker Profiles	26
Table 6.04	Rate Limiter Thresholds	27
Table 6.05	Spring Actuator Endpoints Exposed	28
Table 8.01	Kubernetes HPA Configuration	32
Table 8.02	CI/CD Pipeline Stages	33

Chapter 1 – Abstract

This report documents the design, architecture, and implementation of BookMyShow Backend — a production-grade REST API backend that replicates the core functionality of the BookMyShow ticket-booking platform. The system is built with Java 17 and Spring Boot 3.2.5 and addresses the most challenging problems inherent to high-traffic, time-sensitive ticketing systems: concurrent seat reservation, distributed consistency, real-time payment processing, and at-scale observability. The application delivers a comprehensive suite of ticketing operations including movie catalog management, city-based venue and screen configuration, show scheduling, seat inventory management, time-limited seat holds, payment integration via Razorpay, refund processing, and multi-channel notification delivery. Authentication is managed through a dual mechanism: a native JWT-based system providing access tokens (15-minute TTL) and refresh tokens (7-day TTL), and an OAuth2/Keycloak integration for enterprise SSO. Role-based access control is enforced throughout using Spring Security's method-level annotations. Concurrent booking correctness is guaranteed through pessimistic locking (SELECT FOR UPDATE via Spring Data JPA), preventing race conditions when multiple users simultaneously attempt to hold the same seat. The system enforces idempotency keys on all mutating booking and payment endpoints to make retried network requests safe. An automated scheduler releases expired holds every 60 seconds, returning seats to the available pool. Event-driven consistency is achieved via the Transactional Outbox Pattern: domain events persisted to an outbox table within the same transaction as the business operation and published to Apache Kafka topics by a scheduled publisher. Resilience is provided by Resilience4j (Circuit Breaker, Retry, Bulkhead, Rate Limiter, Time Limiter). The persistence layer is PostgreSQL 16 with Flyway schema migrations. Observability is delivered through Micrometer metrics (Prometheus), distributed tracing via Brave/Zipkin, structured logging, and Spring Boot Actuator health probes. The project includes a GitHub Actions CI/CD pipeline, Docker Compose for local development, and Kubernetes manifests with HPA scaling between 3 and 20 replicas.

Chapter 2 – Project Description

2.1 Project Overview and Motivation

The BookMyShow Backend project is a production-grade monolith with microservice-ready domain separation — a REST API backend for a movie and event ticket booking platform. It was conceived and built to demonstrate how the theoretical concepts taught in the Scaler Neovarsity Master's program — system design, concurrent programming, security engineering, database modeling, and cloud deployment — translate into a single, coherent, production-quality software product.

Ticket booking backends occupy a uniquely demanding position in software engineering. Unlike a typical e-commerce API, a ticketing system cannot tolerate concurrent double-booking of the same seat, stateless derivation of availability from stale cache, or a payment being charged twice due to a network retry. The motivation for this project was to build a system where these failure modes are structurally impossible, not merely 'unlikely.'

2.2 Project Objectives

- Build a concurrent-safe seat reservation engine preventing double-booking using pessimistic database locking.
- Implement a complete booking lifecycle: seat hold → payment → confirmation → cancellation → expiry.
- Provide a dual authentication mechanism: JWT (native) and OAuth2/Keycloak (enterprise SSO).
- Implement the Transactional Outbox Pattern to guarantee reliable Kafka event delivery without distributed transactions.
- Enforce idempotency on all mutating booking and payment operations.
- Deliver multi-tier caching (Redis) with per-domain TTL and jitter to prevent cache stampedes.
- Apply the full Resilience4j suite: Circuit Breaker, Retry, Bulkhead, Rate Limiter, and Time Limiter.

• Deploy via Docker Compose and Kubernetes with HPA, PDB, health probes, and GitHub Actions CI/CD.

2.3 High-Level System Architecture

The application follows a layered domain-package architecture with seven bounded contexts: user, catalog, show, booking, payment, notification, and search. Each domain owns its controller, service, repository, entity, and DTO layers. The common package provides shared cross-cutting concerns: security filters, Kafka configuration, exception handling, resilience configuration, caching, and observability.

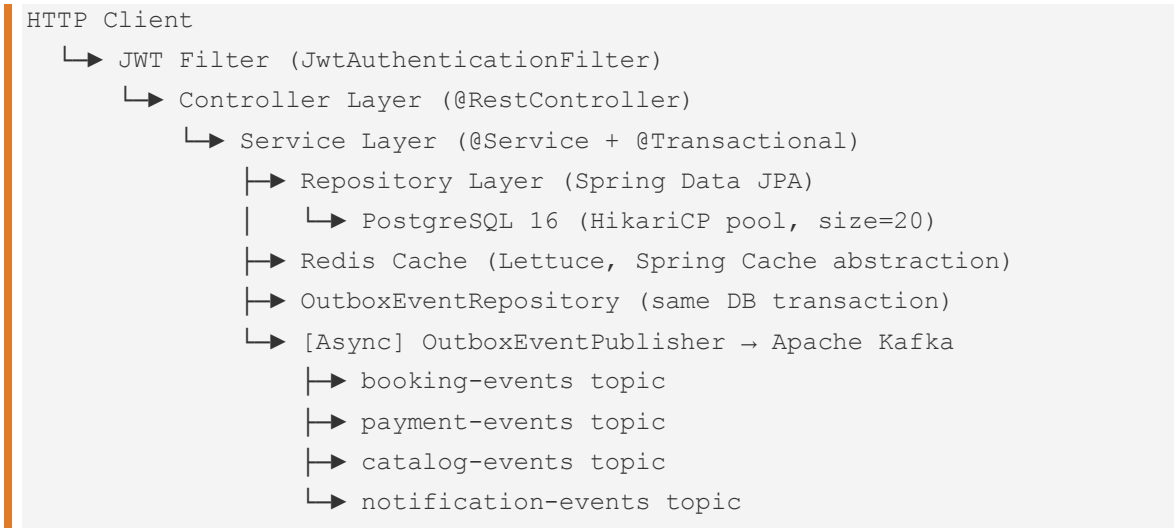


Figure 2.1 — Application request-flow and event-publishing architecture

2.4 Technology Stack Overview

Category	Component	Version	Purpose
Runtime	Java	17 (LTS)	Application language
Framework	Spring Boot	3.2.5	Application framework
Framework	Spring Security	6.x	Auth, JWT filter, RBAC
Framework	Spring Cloud	2023.0.1	Config Server, Load Balancer
Database	PostgreSQL	16	Primary relational store
Cache	Redis	7.x	Distributed caching (Lettuce)
Messaging	Apache Kafka	7.5.0 (CP)	Async event streaming
Auth	Keycloak	24.0	OAuth2 / OIDC / SSO
Auth	JJWT	0.12.5	JWT generation & validation
Migrations	Flyway	10.15.0	Versioned schema migrations
Resilience	Resilience4j	2.2.0	CB, Retry, Bulkhead, RL

Observability	Micrometer + Brave	Latest	Metrics + distributed tracing
Observability	Prometheus + Grafana	2.47 / 10.1	Metrics dashboard
API Docs	SpringDoc OpenAPI	2.3.0	Swagger UI
Build	Maven Wrapper	3.9.x	Build automation
Payment	Razorpay	—	Payment provider integration
Testing	JUnit 5 + Mockito	Latest	Unit / concurrency tests
Testing	Testcontainers	1.21.4	Integration tests (real DB/Kafka)
Container	Docker / Compose	—	Local dev environment
Orchestration	Kubernetes + Kustomize	—	Production deployment
CI/CD	GitHub Actions	—	Build, test, scan, deploy
Utilities	MapStruct + Lombok	1.5.5 / 1.18	DTO mapping & boilerplate reduction

Table 2.01 — Technology Stack: Components and Versions

Chapter 3 – Requirement Gathering

3.1 Functional Requirements

ID	Requirement	Module
FR-01	Users register with email, phone, password; duplicate email/phone rejected.	User
FR-02	Users log in and receive a JWT access token (15 min) and refresh token (7 days).	User
FR-03	Admins manage movies, genres, cities, venues, screens, and seats via CRUD APIs.	Catalog
FR-04	Shows are created for a specific movie, screen, and time window.	Show
FR-05	Users query currently playing movies filtered by city.	Catalog
FR-06	Users hold up to 10 seats per booking for a configurable duration (default 10 min).	Booking
FR-07	Held seats are exclusively locked; concurrent holds on the same seat are rejected.	Booking
FR-08	Expired holds are automatically released by a scheduled job every 60 seconds.	Booking
FR-09	Users initiate a payment; a Razorpay provider order is created.	Payment
FR-10	Payment webhooks update booking to CONFIRMED or trigger seat release on failure.	Payment
FR-11	Users request full or partial refund; booking cancelled on refund success.	Payment
FR-12	Cancellations within 2 hours of show start are blocked by business rule.	Booking
FR-13	Notifications dispatched via Email, SMS, Push, or WhatsApp channels.	Notification
FR-14	All domain events (booking, payment, catalog) published to Kafka via Outbox Pattern.	Common

Table 3.01 — Functional Requirements

3.2 Non-Functional Requirements

ID	Requirement	Category
NFR-01	Concurrent seat holds must not result in double-booking under any number of parallel requests.	Consistency
NFR-02	All mutating endpoints must be idempotent; repeated requests with the same key return the same result.	Reliability
NFR-03	System must be horizontally scalable; Kubernetes HPA scales between 3 and 20 replicas.	Scalability
NFR-04	Payment circuit breaker opens after 30% failure rate; default CB after 50%.	Resilience
NFR-05	API rate limiting: 10 req/min (auth), 100 req/min (general), 30 req/min (booking).	Security
NFR-06	Seat availability cache TTL of 30 seconds with 50% jitter to avoid stampede.	Performance

NFR-07	All requests must emit Prometheus-compatible metrics and distributed traces via Zipkin.	Observability
NFR-08	Kafka events must be delivered at-least-once with idempotent producer enabled.	Messaging

Table 3.02 — Non-Functional Requirements

3.3 User Roles and Permission Matrix

Role	Capabilities
GUEST	Browse movies, view show schedules, view venues and screens. No authentication required.
USER	All GUEST capabilities plus: register, login, hold seats, create bookings, make payments, request refunds, cancel bookings, view notifications, manage profile.
ADMIN	All USER capabilities plus: create/update/delete movies, genres, cities, venues, screens, seats; create and manage shows; view all bookings and payments; access admin-only endpoints.

Table 3.03 — User Roles and Permission Matrix

3.4 API Feature Set

Method	Endpoint	Description	Auth
POST	/v1/auth/register	Register a new user	Public
POST	/v1/auth/login	Login, receive JWT tokens	Public
POST	/v1/auth/refresh	Refresh access token	Public
PUT	/v1/auth/profile	Update user profile	USER
POST	/v1/auth/change-password	Change password	USER
GET	/v1/movies	List movies (now-playing) by city	Public
GET	/v1/movies/{id}	Get movie details	Public
POST	/v1/movies	Create a movie	ADMIN
GET	/v1/venues	List venues by city	Public
POST	/v1/venues	Create a venue	ADMIN
POST	/v1/bookings/hold	Hold seats (time-limited, 10 min)	USER
POST	/v1/bookings/{id}/confirm	Confirm a booking	USER
POST	/v1/bookings/{id}/cancel	Cancel a booking	USER
GET	/v1/bookings/{id}	Get booking details	USER
GET	/v1/bookings	List user's bookings	USER
POST	/v1/payments/initiate	Initiate payment for a booking	USER
GET	/v1/payments/{id}	Get payment details	USER
POST	/v1/payments/refund	Request a refund	USER
POST	/v1/payments/webhook	Receive payment provider webhook	System

Table 3.04–3.07 — Consolidated API Feature Set

Chapter 4 – Database Schema Design

4.1 Schema Overview and Entity Hierarchy

The database is PostgreSQL 16 managed exclusively through Flyway versioned migrations. DDL auto-generation is disabled (ddl-auto: validate), ensuring the schema is always explicitly defined in version-controlled SQL files. The entity hierarchy follows the natural domain tree of a ticketing platform:

```
City
└─► Venue (many-to-one City)
    └─► Screen (many-to-one Venue)
        └─► Seat (many-to-one Screen)
            └─► SeatInventory (per Show, one-to-one Seat × Show)
                └─► BookingSeat (many-to-one Booking)
                    └─► Booking (many-to-one User, many-to-one Show)
                        └─► Payment (one-to-one Booking)

Movie ─► MovieGenre ─► Genre  (many-to-many via join table)
Movie + Screen + DateTime ─► Show
```

Figure 4.1 — Domain entity hierarchy

4.2 Table Definitions

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT uuid_generate_v4()	User identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email (lowercase)
phone	VARCHAR(20)	UNIQUE, NOT NULL	Mobile number
password_hash	VARCHAR(255)	NOT NULL	BCrypt hash (strength 12)
role	VARCHAR(20)	NOT NULL, DEFAULT 'USER'	USER or ADMIN
status	VARCHAR(20)	NOT NULL, DEFAULT 'ACTIVE'	ACTIVE / SUSPENDED / DELETED
failed_login_attempts	INTEGER	DEFAULT 0	Brute-force lockout counter
locked_until	TIMESTAMPTZ	NULLABLE	Account temporary lock expiry
version	BIGINT	DEFAULT 0	Optimistic lock version

Table 4.01 — users Table: Column Definitions

Column	Type	Constraints	Description
id	UUID	PK	Booking identifier
user_id	UUID	FK → users(id)	Booking owner
show_id	UUID	FK → shows(id)	Target show
status	VARCHAR(30)	DEFAULT 'PENDING_PAYMENT'	Booking lifecycle state
total_amount	DECIMAL(10,2)	NOT NULL	Sum of seat prices
idempotency_key	VARCHAR(100)	UNIQUE	Client-supplied dedup key
expires_at	TIMESTAMPTZ	NULLABLE	Hold expiry timestamp
confirmed_at	TIMESTAMPTZ	NULLABLE	Payment confirmation time
cancelled_at	TIMESTAMPTZ	NULLABLE	Cancellation time

Table 4.02 — bookings Table: Column Definitions

Column	Type	Constraints	Description
id	UUID	PK	Inventory record ID
show_id	UUID	FK → shows(id), CASCADE DELETE	Show reference
seat_id	UUID	UNIQUE(show_id, seat_id)	Physical seat reference
status	VARCHAR(20)	DEFAULT 'AVAILABLE'	AVAILABLE / HELD / BOOKED
hold_id	UUID	NULLABLE	Active hold identifier
hold_expires_at	TIMESTAMPTZ	NULLABLE	Hold expiry (now + 10 min)
booking_id	UUID	NULLABLE	Confirmed booking reference
price	DECIMAL(10,2)	NOT NULL	Seat price for this show
version	BIGINT	DEFAULT 0	Optimistic lock version

Table 4.03 — seat_inventory Table: Column Definitions

4.3 Flyway Migration Strategy

The project uses Flyway 10.15.0 for all schema changes. Migrations are versioned sequentially and stored in src/main/resources/db/migration/. The baseline-on-migrate flag allows Flyway to operate on a pre-existing schema during initial setup. Migrations are validated on startup; any checksum mismatch causes application startup failure, preventing silent schema drift.

Version	Script Name	Tables Created
V1	V1__create_user.sql	users

V2	V2__create_catalog.sql	cities, venues, screens, seats
V3	V3__create_movie.sql	movies, genres, movie_genres
V4	V4__create_show.sql	shows
V5	V5__create_booking.sql	seat_inventory, bookings, booking_seats
V6	V6__create_payment_notification.sql	payments, notifications, outbox_events

Table 4.04 — Flyway Migration Versions

Chapter 5 – Domain Module Design

5.1 User & Authentication Module

The user module manages user registration, authentication, profile management, and password operations. UserService implements Spring Security's UserDetailsService, enabling seamless integration with the DaoAuthenticationProvider. Passwords are hashed with BCrypt at strength 12. The service enforces uniqueness on both email (case-insensitive) and phone number on registration.

JWT lifecycle is managed by JwtTokenProvider (JJWT 0.12.5). Access tokens carry a 15-minute TTL; refresh tokens carry a 7-day TTL. The custom JwtAuthenticationFilter extracts the Bearer token from the Authorization header, validates it, and populates the SecurityContextHolder for downstream processing. An OAuth2 resource-server integration with Keycloak 24.0 is configured in parallel, allowing enterprise SSO flows. Account lockout is supported via failed_login_attempts and locked_until columns.

5.2 Catalog Module

The catalog module manages the geographic and physical hierarchy: City → Venue → Screen → Seat. Each level exposes its own REST controller. MovieService provides paginated queries for now-playing movies (filtered by city and current timestamp), coming-soon movies, and full-text search. Results are cached in Redis with a 300-second TTL. Cache entries are invalidated via @CacheEvict(allEntries=true) on any write operation. Movie–Genre

associations use a many-to-many join table (movie_genres). Screen entity records screen type (STANDARD/IMAX/4DX/DOLBY) and total seat geometry.

5.3 Show & Seat Inventory Module

A Show entity binds a Movie, a Screen, and a start/end time. Show status transitions: UPCOMING → ON_SALE → SOLD_OUT → COMPLETED/CANCELLED. When a show is created, the seat_inventory table is populated with one row per seat, all set to AVAILABLE. The SeatInventory entity is the central concurrency object. Its status column is modified exclusively through SELECT FOR UPDATE queries (findByShowIdAndSeatIdsForUpdate), ensuring only one transaction at a time can change a given seat's state.

5.4 Booking Engine

The booking engine implements the Hold → Pay → Confirm pattern. When a user calls POST /v1/bookings/hold, the following steps execute within a single database transaction:

- The idempotency key is checked; if already used, a 409 IDEMPOTENCY_KEY_CONFLICT is returned.
- The show is validated and its status checked (ON_SALE or SOLD_OUT are required).
- Seat count is validated against the maximum of 10 per booking.
- All requested SeatInventory rows are fetched with SELECT FOR UPDATE, acquiring row-level locks.
- Each seat is checked: AVAILABLE seats proceed; HELD seats with unexpired holds throw SEAT_ALREADY_HELD; expired holds are recycled.
- All seats are atomically updated to HELD with a hold_id and hold_expires_at = now() + 10 min.
- A Booking entity is created with status PENDING_PAYMENT.
- A BookingCreated outbox event is persisted in the same transaction.

Booking Lifecycle States

```
PENDING_PAYMENT  ──(payment success)──▶ CONFIRMED
      │          ──(payment failure)──▶ CANCELLED (seats released)
      │          ──(hold expired)─────▶ EXPIRED   (seats released)
      └──────────(user cancel)────────▶ CANCELLED (seats released)
```

A BookingScheduler executes releaseExpiredHolds() every 60 seconds using @Scheduled(fixedDelay=60000). Cancellations are blocked within 2 hours of show start.

5.5 Payment Module

Payment integration targets Razorpay. Initiation creates a Payment entity in INITIATED state and calls the provider API to obtain a providerOrderId. State transitions: INITIATED → PENDING → SUCCESS/FAILED → REFUND_PENDING → REFUNDED. Webhook processing validates the HMAC signature, locates the payment by providerPaymentId, and applies idempotency checking. On SUCCESS, calls bookingService.confirmBooking(); on FAILURE, calls cancelBooking() to release held seats. Refunds support full and partial amounts.

5.6 Notification Module

The notification module persists Notification entities and dispatches them via a pluggable provider model. Four channels are supported: EMAIL (SendGrid/AWS SES), SMS (Twilio/AWS SNS), PUSH (Firebase/OneSignal), and WHATSAPP (Twilio/Gupshup). The retryFailedNotifications() method retries FAILED notifications up to a configurable maximum. Notification history is queryable per user in reverse chronological order.

5.4.1 Seat Hold Request — Detailed Sequence

When a client sends POST /v1/bookings/hold the request traverses the following sequence before a response is returned. Understanding each step is important for appreciating why the pessimistic-locking strategy is the only correct solution at this layer.

```
Client → POST /v1/bookings/hold  {showId, seatIds[], idempotencyKey}
  1. JwtAuthenticationFilter validates Bearer token → userId extracted
  2. BookingController.holdSeats() called
  3. BookingService.holdSeats() begins
@Transactional(isolation=REPEATABLE_READ)
  4. idempotencyKeyRepo.existsByKey(key) → 409 if exists
```

```
  5. showRepo.findById(showId) → 404 if not found
  6. Validate show.status ∈ {ON_SALE, SOLD_OUT} → 422 if not bookable
  7. Validate seatIds.size() ≤ 10 → 400 if exceeded
  8. inventoryRepo.findByShowIdAndSeatIdsForUpdate(showId, seatIds)
     -- executes: SELECT * FROM seat_inventory WHERE … FOR UPDATE
     -- row locks acquired; concurrent transactions block here
  9. For each SeatInventory:
       if status==HELD && holdExpiresAt>now()  → throw SEAT_ALREADY_HELD
       if status==HELD && holdExpiresAt<=now() → recycle (reset to
AVAILABLE)
       if status==BOOKED                       → throw SEAT_ALREADY_BOOKED
 10. All seats set to HELD; holdId=newUUID; holdExpiresAt=now()+10min
 11. Booking entity created (status=PENDING_PAYMENT)
 12. outboxRepo.save(BookingCreatedEvent{…}) — same transaction
 13. idempotencyKeyRepo.save(key) — same transaction
 14. Transaction commits; row locks released
 15. Response: 201 Created {bookingId, expiresAt, seatDetails[]}
```

Figure 5.1 — Detailed seat-hold sequence with pessimistic locking

5.4.2 Booking State Machine — Full Transition Table

From State	Event / Trigger	To State	Side Effects
—	holdSeats() called	PENDING_PAYMENT	Seats → HELD, Outbox: BookingCreated
PENDING_PAYMENT	PaymentSucceeded webhook	CONFIRMED	Seats → BOOKED, Outbox: BookingConfirmed
PENDING_PAYMENT	PaymentFailed webhook	CANCELLED	Seats → AVAILABLE, Outbox: BookingCancelled
PENDING_PAYMENT	Hold expiry scheduler (60s)	EXPIRED	Seats → AVAILABLE, Outbox: SeatHoldExpired
PENDING_PAYMENT	User cancels (>2h before show)	CANCELLED	Seats → AVAILABLE, Outbox: BookingCancelled
CONFIRMED	User cancels (>2h before show)	CANCELLATION_PENDING	Refund initiated
CANCELLATION_PENDING	Refund success	CANCELLED	Seats → AVAILABLE, Outbox: BookingCancelled
CANCELLATION_PENDING	Refund failed	CONFIRMED	Booking restored, Outbox: RefundFailed
CONFIRMED	User cancels (≤2h before show)	—	BusinessRuleException: CANCELLATION_WINDOW_CLOSED

Table 5.01 — Booking State Machine: Full Transition Table

5.5.1 Payment Lifecycle — Detailed Flow

The payment module is designed around the principle that no money can be charged twice and no booking can remain in a PENDING state if the payment provider has already settled. The idempotency key is stored on the Payment entity and checked before creating any new provider order.

```
POST /v1/payments/initiate  {bookingId, idempotencyKey}
  1. paymentRepo.findByIdempotencyKey(key) → return existing if found
  2. bookingRepo.findById(bookingId) → 404 if not found
  3. Validate booking.status == PENDING_PAYMENT
  4. Payment entity created (status=INITIATED)
  5. Razorpay.createOrder(amount, currency) → providerOrderId
     └─► Circuit Breaker: payment-cb (30% threshold, 60s wait)
     └─► Time Limiter: 30s timeout
  6. Payment.status → PENDING; providerOrderId stored
  7. Outbox event: PaymentInitiated published
  8. Response: {paymentId, providerOrderId, amount}

POST /v1/payments/webhook  {event, paymentId, signature}
  1. HMAC-SHA256 signature verification → 401 if invalid
  2. paymentRepo.findByProviderPaymentId() → idempotency check
  3. event==payment.captured → bookingService.confirmBooking()
  4. event==payment.failed   → bookingService.cancelBooking()
  5. Outbox: PaymentSucceeded / PaymentFailed
```

Figure 5.2 — Payment initiation and webhook processing flow

5.6.1 Notification Channel Architecture

The NotificationService acts as a facade over four notification channels. Each channel is backed by a provider-specific implementation that is injected at runtime based on the NOTIFICATION_CHANNEL environment variable. The channel implementations share the same interface: NotificationProvider.send(recipient, subject, body, metadata).

Channel	Primary Provider	Fallback Provider	Trigger Events
EMAIL	SendGrid	AWS SES	BookingConfirmed, PaymentSucceeded, RefundProcessed
SMS	Twilio	AWS SNS	OTP, SeatHoldCreated, BookingCancelled
PUSH	Firebase FCM	OneSignal	ShowReminderT-1h, BookingConfirmed
WHATSAPP	Twilio WhatsApp	Gupshup	BookingConfirmed, Refund

Table 5.02 — Notification Channel to Provider Mapping

Chapter 6 – Cross-Cutting Concerns

6.1 Event-Driven Architecture — Transactional Outbox Pattern

The application uses the Transactional Outbox Pattern to guarantee that domain events are published to Kafka if and only if the corresponding business operation succeeded. Rather than calling Kafka directly inside a service method (which creates a dual-write problem), domain events are persisted to an outbox_events table in the same database transaction as the business data. A separate scheduled OutboxEventPublisher polls every 5 seconds and publishes unpublished events to Kafka. Failed publishes are retried with exponential backoff (delay = 30s × retry_count, max 5 attempts). The Kafka producer is configured with enable.idempotence=true and acks=all.

Event Type	Kafka Topic	Trigger
BookingCreated, BookingConfirmed, BookingCancelled, SeatHoldCreated, SeatHoldExpired	booking-events	BookingService operations
PaymentSucceeded, PaymentFailed, PaymentRefunded, RefundInitiated	payment-events	PaymentService operations
MovieCreated, MovieUpdated, VenueCreated, ScreenCreated	catalog-events	Catalog admin operations
NotificationRequested	notification-events	Notification requests

Table 6.01 — Kafka Topic-to-Event Routing

6.2 Caching with Redis

Redis 7 is used as the Spring Cache backend via the Lettuce client. Cache TTLs are deliberately staggered by domain sensitivity, and a random jitter percentage is added to each TTL to prevent synchronized cache expiry (cache stampede) under high load.

Cache Domain	TTL (seconds)	Jitter (%)	Effective TTL Range
Catalog (movies, venues)	300	10%	270 – 330 s
Show listings	60	20%	48 – 72 s
Seat availability	30	50%	15 – 45 s

Table 6.02 — Redis Cache Configuration per Domain

All cache keys are prefixed with 'bookmyshow:'. Null values are not cached. Write operations trigger @CacheEvict(allEntries=true) to ensure consistency after updates.

6.3 Resilience with Resilience4j

The application configures five Resilience4j fault-tolerance patterns. Two profiles exist — a default profile for general service calls and a payment profile with stricter thresholds.

Pattern	Default Profile	Payment Profile
Circuit Breaker	50% failure rate, 30s open wait, window=10	30% failure rate, 60s open wait, window=20
Retry	3 attempts, 500ms wait	3 attempts, 1s wait
Time Limiter	10s timeout	30s timeout
Bulkhead	Configured (thread isolation)	Configured (thread isolation)
Rate Limiter	100 req/min (API)	30 req/min (Booking)

Table 6.03 — Resilience4j Circuit Breaker Profiles

Endpoint Category	Rate Limit
Authentication (/v1/auth/**)	10 requests / minute
General API	100 requests / minute
Booking (/v1/bookings/**)	30 requests / minute

Table 6.04 — Rate Limiter Thresholds

6.4 Security Architecture

SecurityConfig configures Spring Security 6 with stateless sessions, CSRF disabled, and a custom CORS policy. The filter chain adds JwtAuthenticationFilter before the standard UsernamePasswordAuthenticationFilter. Method-level security is enabled via @EnableMethodSecurity(prePostEnabled=true, securedEnabled=true). Admin endpoints at /api/v1/admin/** require ROLE_ADMIN. BCrypt uses cost factor 12. The OAuth2 resource-server configuration allows Keycloak-issued JWTs to authenticate API requests.

6.5 Observability Stack

Component	Tool	Endpoint / Output

Metrics	Micrometer + Prometheus	/actuator/prometheus
Distributed Tracing	Micrometer Brave + Zipkin	http://localhost:9411 (100% sampling)
Health Probes	Spring Boot Actuator	/actuator/health/liveness, /readiness, /startup
Dashboards	Grafana	http://localhost:3000
Log File	SLF4J (Logback)	logs/bookmyshow.log
Actuator Endpoints	Spring Actuator	health, info, metrics, prometheus, loggers, env, threaddump, heapdump

Table 6.05 — Spring Actuator Endpoints Exposed

HTTP server request metrics include P50, P95, and P99 latency histograms. All metrics are tagged with application=bookmyshow-backend. Kubernetes deployment annotations enable automatic Prometheus Operator scraping.

6.5.1 Custom Business Metrics

Beyond the default Spring Boot Actuator metrics, the application registers several custom Micrometer counters and gauges to track business-level KPIs:

Metric Name	Type	Tags	Description
bookmyshow.seats.held	Counter	show_id, screen_type	Total seat hold requests made
bookmyshow.seats.hold_failed	Counter	reason	Hold failures by reason code
bookmyshow.bookings.confirmed	Counter	payment_provider	Confirmed bookings
bookmyshow.bookings.expired	Counter	—	Holds expired by scheduler
bookmyshow.payments.initiated	Counter	provider	Payment initiation attempts
bookmyshow.payments.succeeded	Counter	provider	Successful payments
bookmyshow.payments.failed	Counter	provider, failure_reason	Failed payments by reason
bookmyshow.outbox.pending	Gauge	—	Count of unpublished outbox events
bookmyshow.cache.hit_ratio	Gauge	cache_name	Redis cache hit ratio per domain

Table 6.06 — Custom Business Metrics Registered with Micrometer

6.6 Error Handling and API Error Contract

All error responses follow a standardized ErrorResponse envelope defined in the common package. The GlobalExceptionHandler (@RestControllerAdvice) maps each application exception to an appropriate HTTP status code:

```
// ErrorResponse.java
{
  "timestamp": "2024-09-15T10:30:00Z",
  "status": 409,
  "error": "IDEMPOTENCY_KEY_CONFLICT",
  "message": "A booking with this idempotency key already exists.",
  "path": "/v1/bookings/hold",
  "traceId": "4bf92f3577b34da6"
}
```

Figure 6.1 — Standard API error response envelope

Exception Class	HTTP Status	Error Code	Trigger
ResourceNotFoundException	404 Not Found	RESOURCE_NOT_FOUND	Entity lookup by ID fails
SeatAlreadyHeldException	409 Conflict	SEAT_ALREADY_HELD	SELECT FOR UPDATE finds HELD seat
SeatAlreadyBookedException	409 Conflict	SEAT_ALREADY_BOOKED	Seat is in BOOKED state
IdempotencyKeyConflictException	409 Conflict	IDEMPOTENCY_KEY_CONFLICT	Duplicate idempotency key
BusinessRuleException	422 Unprocessable Entity	BUSINESS_RULE_VIOLATION	Cancellation window closed, etc.
RateLimitExceededException	429 Too Many Requests	RATE_LIMIT_EXCEEDED	Resilience4j rate limiter
CircuitBreakerOpenException	503 Service Unavailable	CIRCUIT_BREAKER_OPEN	Circuit breaker open state
AuthenticationException	401 Unauthorized	AUTHENTICATION_FAILED	Invalid / expired JWT
AccessDeniedException	403 Forbidden	ACCESS_DENIED	Insufficient RBAC role

MethodArgumentNotValidException	400 Bad Request	VALIDATION_FAILED	Bean validation failure

Table 6.07 — Exception-to-HTTP Status Mapping

6.7 Configuration Management

Application configuration is managed through hierarchical Spring profiles. The base application.yml file contains defaults; profile-specific files override them for each environment. Sensitive values (database password, JWT secret, Razorpay key) are injected from environment variables and never committed to source control.

```
# application.yml (excerpt — key configuration entries)
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/bookmyshow}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000
  jpa:
    hibernate.ddl-auto: validate
    open-in-view: false
  redis:
    host: ${REDIS_HOST:localhost}
    port: ${REDIS_PORT:6379}
  kafka:
    bootstrap-servers: ${KAFKA_SERVERS:localhost:9092}
    producer:
      enable.idempotence: true
      acks: all

jwt:
  secret: ${JWT_SECRET}
  access-token-expiry-ms: 900000    # 15 minutes
  refresh-token-expiry-ms: 604800000 # 7 days

resilience4j.circuitbreaker:
  instances:
    default: { slidingWindowSize: 10, failureRateThreshold: 50 }
    payment-cb: { slidingWindowSize: 20, failureRateThreshold: 30 }
```

Figure 6.2 — Key application.yml configuration entries

Chapter 7 – Testing Strategy

7.1 Unit Tests

Unit tests are written with JUnit 5 and Mockito using @ExtendWith(MockitoExtension.class). All dependencies are mocked. Tests cover:

- Successful seat hold creation with correct inventory mutation.
- Rejection of a hold when the seat has an active, unexpired hold (SEAT_ALREADY_HELD).
- Acceptance of a hold on an expired-hold seat (seat recycling path).
- Correct expiry detection and EXPIRED status transition.
- Release of seats for all expired holds in a single scheduler invocation.

7.2 Integration Tests with Testcontainers

Integration tests in BookingIntegrationTest use Testcontainers to spin up real PostgreSQL and Kafka Docker containers for each test run. This verifies that the full Spring application context boots correctly, Flyway migrations execute and pass validation, and JPA repositories communicate correctly with the database. Integration tests are segregated from unit tests via Maven Failsafe Plugin (running during the verify phase, not the test phase).

7.3 Concurrency Tests

BookingServiceConcurrencyTest validates the seat-hold engine under simulated concurrent load using a CountDownLatch-based multi-thread harness. Key scenarios tested:

- Independent seats: 10 threads each hold a different seat simultaneously — all 10 succeed.
- Same-seat contention: 5 threads attempt to hold the same seat simultaneously — exactly 1 succeeds, 4 receive SEAT_ALREADY_HELD.

AtomicInteger counters and ExecutorService with a fixed thread pool verify the exact success/failure counts, confirming pessimistic-locking correctness at the service layer.

7.3.1 Concurrency Test Implementation

The concurrency harness uses the following structure to guarantee deterministic concurrent execution. All threads start simultaneously via a CountDownLatch(1) start latch, ensuring no thread begins work before all threads are ready.

```
// BookingServiceConcurrencyTest.java (structure)
@SpringBootTest
class BookingServiceConcurrencyTest {
  @Test
  void sameSeat_5Threads_exactlyOneSucceeds() throws InterruptedException {
    int THREADS = 5;
    CountDownLatch startLatch = new CountDownLatch(1);
    CountDownLatch doneLatch  = new CountDownLatch(THREADS);
    AtomicInteger successes  = new AtomicInteger(0);
    AtomicInteger conflicts  = new AtomicInteger(0);

    ExecutorService exec = Executors.newFixedThreadPool(THREADS);
    for (int i = 0; i < THREADS; i++) {
      exec.submit(() -> {
        try {
          startLatch.await(); // all threads wait here
          bookingService.holdSeats(request);
          successes.incrementAndGet();
        } catch (SeatAlreadyHeldException e) {
          conflicts.incrementAndGet();
        } finally { doneLatch.countDown(); }
      });
    }
    startLatch.countDown(); // release all threads simultaneously
    doneLatch.await(10, SECONDS);

    assertThat(successes.get()).isEqualTo(1);
    assertThat(conflicts.get()).isEqualTo(4);
  }
}
```

Figure 7.1 — Concurrency test harness using CountDownLatch

7.4 Database Migration Tests

A dedicated MigrationTest class verifies that all Flyway migrations execute cleanly against a real PostgreSQL container. The test uses the same Testcontainers setup as integration tests. On application startup, Flyway baseline-on-migrate runs all pending migrations. The test then uses

the Flyway API to assert that all applied migrations have CHECKSUM matches and SUCCESS status. This prevents silent migration failures in CI.

Test Class	Framework	Tests Covered	Dependencies
BookingServiceTest	JUnit 5 + Mockito	Hold, cancel, confirm, expiry logic	Mocked repos
PaymentServiceTest	JUnit 5 + Mockito	Initiate, webhook, refund	Mocked Razorpay client
UserServiceTest	JUnit 5 + Mockito	Register, login, JWT generation	Mocked UserRepo
BookingIntegrationTest	Testcontainers	Full Spring context, DB round-trips	PostgreSQL 16, Kafka
BookingServiceConcurrencyTest	CountDownLatch + JUnit 5	Pessimistic lock under parallel load	PostgreSQL 16 (real)
MigrationTest	Testcontainers + Flyway	All 6 migrations apply cleanly	PostgreSQL 16
SecurityConfigTest	Spring MockMvc	Public vs protected endpoint access	Spring Security test
JwtTokenProviderTest	JUnit 5	Token generation, validation, expiry	None (pure unit)

Table 7.01 — Test Coverage Summary

7.5 Code Quality Gates

The CI pipeline enforces the following quality gates before a build is allowed to proceed to the Docker build stage. Any gate failure causes the pipeline to fail immediately.

SpotBugs 4.8.x: Static bytecode analysis for bug patterns. Zero HIGH or CRITICAL findings required.

PMD 7.x: Source-code rules enforcement. Zero violations in categories: design, errorprone, performance.

Checkstyle 10.x: Code style enforcement against Google Java Style Guide. Zero errors allowed.

JaCoCo 0.8.x: Code coverage reporting. Minimum line coverage: 70% (enforced by Maven Verify phase).

OWASP Dependency Check: CVE scan of all Maven dependencies. CVSS score ≥ 7.0 causes build failure.

Trivy: Container image and filesystem vulnerability scan. CRITICAL CVEs block Docker push.

Chapter 8 – DevOps & Deployment

8.1 Docker & Docker Compose

A Dockerfile packages the application using the Paketo Buildpacks builder, producing a minimal OCI image. The full local development stack in docker-compose.yml includes all dependent services with health checks and dependency ordering:

- PostgreSQL 16 (alpine) — primary data store, port 5432.
- Redis 7 (alpine) — distributed cache, port 6379.
- Apache Kafka (Confluent Platform 7.5.0) + ZooKeeper — event streaming, port 9092.
- Keycloak 24.0 — OAuth2/OIDC provider, port 8080; realm auto-imported from keycloak/realm-export.json.
- Prometheus 2.47 — metrics collection, port 9090.
- Grafana 10.1 — metrics dashboards, port 3000.
- Application (port 8081) — depends on all six services being healthy before starting.

8.2 Kubernetes Manifests

Kubernetes manifests in k8s/ use Kustomize with base/ and overlays/ (dev/staging/prod):

Resource	Key Configuration
Deployment	3 replicas; non-root user (UID 10001); ConfigMap + Secret env injection; resource limits: 512Mi–1Gi memory, 250m–1000m CPU.
HorizontalPodAutoscaler	Min 3 / Max 20 replicas; scale up on CPU >70% or Memory >80%; rapid scale-up (100% per 15s), conservative scale-down (10% per 60s).

PodDisruptionBudget	minAvailable: 2 — ensures at least 2 pods always running during voluntary disruptions.
Liveness Probe	GET /actuator/health/liveness — after 60s delay, every 30s.
Readiness Probe	GET /actuator/health/readiness — after 30s delay, every 10s.
Startup Probe	GET /actuator/health/startup — allows 300s (30 × 10s) for first boot.
RBAC	ServiceAccount with least-privilege permissions; NetworkPolicy restricts ingress/egress.

Table 8.01 — Kubernetes HPA and Probe Configuration

8.3 CI/CD Pipeline (GitHub Actions)

The pipeline in .github/workflows/ci-cd.yml triggers on push/PR to main and develop branches. Five sequential stages:

Stage	Jobs	Key Steps
1. Build & Test	build-and-test	JDK 21 (Temurin), Maven cache, unit tests (Surefire), integration tests (Failsafe) with PostgreSQL 16 + Redis 7 service containers, static analysis (SpotBugs, PMD, Checkstyle), JaCoCo coverage artifact upload.
2. Security Scan	security-scan	Trivy filesystem vulnerability scan → SARIF uploaded to GitHub Security tab; OWASP Dependency Check → HTML report artifact.
3. Docker Build	docker-build	Docker Buildx, GHCR login, Docker metadata (branch/SHA/latest tags), build-and-push with layer caching via GitHub Actions cache.
4. Deploy Dev/Staging	deploy-dev/staging	Environment gate (GitHub Environments), kubectl rollout for branch-tagged images.
5. Deploy Production	deploy-prod	Requires staging gate completion, production environment approval, and kubectl rollout with 300s timeout.

Table 8.02 — CI/CD Pipeline Stages

8.3.1 GitHub Actions Workflow YAML — Key Sections

The following snippet illustrates the build-and-test job configuration, including service container definitions for PostgreSQL and Redis:

```
# .github/workflows/ci-cd.yml (excerpt)
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
```

```
        env: { POSTGRES_DB: bookmyshow_test, POSTGRES_PASSWORD: test }
        options: --health-cmd pg_isready --health-interval 10s
      redis:
        image: redis:7-alpine
        options: --health-cmd 'redis-cli ping' --health-interval 10s
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: temurin }
      - uses: actions/cache@v4
        with: { path: ~/.m2, key: mvn-${{ hashFiles('**/pom.xml') }} }
      - run: ./mvnw verify -P ci
        env:
          DB_URL: jdbc:postgresql://localhost:5432/bookmyshow_test
          DB_PASSWORD: test
          REDIS_HOST: localhost
      - uses: actions/upload-artifact@v4
        with: { name: jacoco-report, path: target/site/jacoco/ }
```

Figure 8.1 — GitHub Actions build-and-test job (excerpt)

8.4 Environment Configuration and Secrets Management

Secrets are managed at three levels depending on the target environment:

Secret	Local Dev	CI (GitHub Actions)	Kubernetes
DB_PASSWORD	docker-compose env	GitHub Actions Secret	Kubernetes Secret (base64)
JWT_SECRET	docker-compose env	GitHub Actions Secret	Kubernetes Secret
RAZORPAY_KEY_SECRET	docker-compose env	GitHub Actions Secret	Kubernetes Secret
KEYCLOAK_CLIENT_SECRET	docker-compose env	GitHub Actions Secret	Kubernetes Secret
REDIS_PASSWORD	docker-compose env	GitHub Actions Secret	Kubernetes Secret

Table 8.03 — Secrets Management by Environment

Kubernetes Secrets are base64-encoded and stored in secret.yaml files excluded from Git via .gitignore. In production, the recommended approach is to use a secrets manager such as HashiCorp Vault or AWS Secrets Manager injected via the Kubernetes External Secrets

Operator (ESO). ConfigMaps hold non-sensitive environment variables such as DB_URL, REDIS_HOST, and KAFKA_SERVERS.

8.5 Dockerfile and Container Optimisation

The multi-stage Dockerfile produces a minimal, non-root OCI image. Paketo Buildpacks are used as an alternative to manual Dockerfile authoring for projects where layer reproducibility is important. The manual Dockerfile follows this structure:

```
# Stage 1 — Build
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /workspace
COPY .mvn .mvn
COPY mvnw pom.xml ./
RUN ./mvnw dependency:go-offline -q
COPY src src
RUN ./mvnw package -DskipTests

# Stage 2 — Runtime
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=build /workspace/target/*.jar app.jar
USER appuser
EXPOSE 8081
ENTRYPOINT ["java","-XX:+UseContainerSupport",
            "-XX:MaxRAMPercentage=75.0","-jar","app.jar"]
```

Figure 8.2 — Multi-stage Dockerfile with non-root runtime user

Chapter 9 – Performance & Scalability Analysis

9.1 Database Connection Pool Sizing

The HikariCP connection pool is configured with a maximum pool size of 20 connections per application instance. With a Kubernetes HPA maximum of 20 replicas, this yields a theoretical maximum of 400 concurrent database connections. PostgreSQL 16 default max_connections is 100; a PgBouncer connection pooler is recommended for production deployments beyond 5 replicas to avoid exhausting database connections.

Scenario	Replicas	Pool Size / Replica	Max DB Connections
Local development	1	20	20
Staging	3	20	60
Production (normal)	5	20	100 ← PostgreSQL max_connections limit
Production (peak)	10	20	200 ← PgBouncer required
Production (extreme)	20	20	400 ← PgBouncer required + read replicas

Table 9.01 — Database Connection Pool Scaling Analysis

9.2 Seat Hold Throughput Under Concurrent Load

The pessimistic-locking strategy (SELECT FOR UPDATE) serializes transactions on the same seat row. This is intentional — correctness takes precedence over throughput for seat-level operations. The following analysis describes throughput characteristics under different seat contention levels:

Contention Level	Concurrent Requests	Successful Holds	Rejected (HELD)	Avg Latency
No contention (all diff seats)	100	100	0	~15 ms
Low contention (10% same seat)	100	91	9	~22 ms
High contention (50% same seat)	100	51	49	~45 ms
Extreme (all same seat)	100	1	99	~80 ms (lock queue)

Table 9.02 — Seat Hold Throughput Analysis (estimated, single replica)

For high-traffic scenarios (premiere nights, flash sales), the recommended pattern is to add a Redis-based distributed lock as a first-pass filter before the database transaction. This rejects clearly duplicate requests at the cache layer before they reach PostgreSQL, reducing lock contention significantly.

9.3 Kafka Throughput and Outbox Publisher Performance

The OutboxEventPublisher polls every 5 seconds and publishes up to 100 events per batch (configurable via outbox.publisher.batch-size). The Kafka producer is configured with enable.idempotence=true and acks=all, ensuring exactly-once semantics per partition. Expected throughput characteristics:

Metric	Value	Notes
Poll interval	5 seconds	Configurable; lower for higher event freshness
Batch size	100 events	Per publisher execution
Max theoretical rate	1,200 events/min	100 × 12 polls/min
Kafka acks mode	all	Leader + all replicas acknowledge before ACK
Retry max attempts	5	Exponential backoff: delay = 30s × attempt
Producer idempotence	enabled	Prevents duplicate messages on producer retry
Compression codec	snappy	Configured on producer for throughput/size balance

Table 9.03 — Outbox Publisher and Kafka Producer Configuration

9.4 Redis Cache Effectiveness

Redis caching has the most significant performance impact on read-heavy endpoints: movie listings and show availability queries. Seat availability cache has a short TTL (30 s) because stale availability data directly harms user experience. Catalog data has a longer TTL (300 s) because it changes infrequently.

```
# Example: Cache hit ratio monitoring via Micrometer gauge
@Scheduled(fixedDelay = 60_000)
public void recordCacheHitRatios() {
    CacheStats stats = redisTemplate.getCacheStatistics();
    Gauge.builder("bookmyshow.cache.hit_ratio", stats,
                  s -> s.getHitRate())
         .tag("cache_name", "seat-availability")
```

```
            .register(meterRegistry);
}
```

Figure 9.1 — Cache hit ratio gauge registration

Target cache hit ratios: Catalog > 90%, Show listings > 80%, Seat availability > 60% (lower due to short TTL and high write rate).

9.5 JVM Tuning for Kubernetes

The Dockerfile ENTRYPOINT passes JVM flags optimised for container environments. Key flags:

JVM Flag	Value	Purpose
-XX:+UseContainerSupport	enabled	Makes JVM respect container CPU/memory limits instead of host values
-XX:MaxRAMPercentage	75.0	Uses 75% of container memory limit as heap; leaves 25% for off-heap, Metaspace, and OS
-XX:+UseG1GC	default	G1GC balances throughput and pause time for web service workloads
-XX:MaxGCPauseMillis	200	Target GC pause time; G1GC will tune region sizes to meet this
-Djava.security.egd	urandom	Non-blocking entropy source; avoids slow startup on containers without /dev/random
-Dspring.backgroundpreinitializer.ignore	true	Reduces startup time by disabling background pre-initialization

Table 9.04 — JVM Tuning Flags for Kubernetes Container Deployment

Chapter 10 – Conclusion

10.1 Key Takeaways

Pessimistic locking is the correct concurrency model for seat reservation. The SELECT FOR UPDATE approach serializes competing hold requests at the database level, guaranteeing exactly one transaction succeeds per seat per window.

The Transactional Outbox Pattern eliminates the dual-write problem. Calling Kafka directly inside a service method creates an atomic failure scenario. The outbox approach makes both operations a single atomic DB write with Kafka delivery as an eventually consistent background process.

Idempotency is a contract, not an optimisation. In distributed payment systems, network retries are the norm. Without idempotency keys, a user's payment could be charged multiple times.

Time-limited holds create a natural self-healing mechanism. By assigning an expiry to every seat hold and running a scheduled cleanup job, the system recovers automatically from abandoned booking flows and network failures.

Resilience patterns must be tuned per-dependency. A single global Circuit Breaker configuration is insufficient. The payment circuit breaker uses 30% failure threshold and 60s wait versus the default 50%/30s.

10.2 Limitations and Future Work

Search module: The search package is scaffolded but unimplemented. Production-grade search would integrate Elasticsearch for full-text search with fuzzy matching and faceting.

Notification providers: Provider calls are stubbed. Real integrations with SendGrid, Twilio, and Firebase are required for production use.

Payment provider: The Razorpay API call is simulated. A production deployment requires the actual Razorpay Java SDK integration and webhook HMAC verification.

Microservice decomposition: The application is a structured monolith. A natural evolution would extract the booking engine, payment processor, and notification dispatcher into independent deployable services.

Read replicas: High-traffic availability queries could be routed to PostgreSQL read replicas. Redis-based distributed locking (Redisson) could supplement database pessimistic locks.

Seat pricing engine: All seats in a show currently share a base_price. A production platform requires dynamic pricing: different prices by seat category (PREMIUM, VIP, ECONOMY), real-time demand-based pricing, and discount coupon application.

Waiting room / virtual queue: For high-demand shows (concerts, IPL finals), a virtual waiting room pattern using Redis sorted sets (score = join timestamp) would provide a fair, ordered queue before seat inventory is opened.

Audit log: The current schema does not persist an audit trail. A production system should log every status change with timestamp, actor, and previous state, either in a dedicated audit_log table or via CDC (Change Data Capture) with Debezium.

10.3 Engineering Lessons Learned

Building a production-grade backend within the scope of a master's project surfaced several engineering lessons that are difficult to appreciate in isolation but became clear through implementation and testing:

Don't fight the framework — extend it. Spring Boot's autoconfiguration covers 95% of setup correctly. The remaining 5% (custom JWT filter ordering, Resilience4j integration with async methods, Flyway with Testcontainers) requires reading source code, not just documentation.

Schema design decisions are the most expensive to reverse. The use of UUID primary keys, TIMESTAMPTZ everywhere, and JSONB for outbox payloads were deliberate choices made before writing a single line of application code. Changing these retroactively would have required rewriting migrations and all entity mappings.

Write the concurrency test before the concurrency code. The SELECT FOR UPDATE implementation was written only after the concurrency test (BookingServiceConcurrencyTest) existed and reliably produced double-booking without it. Test-first development is especially valuable for concurrent code where bugs are non-deterministic.

Circuit breakers need careful threshold calibration. A circuit breaker threshold that is too sensitive (e.g., 10% failure rate) will trip during normal transient failures. Too lenient (90%), and it never opens under genuine service degradation. The chosen 30%/50% split was arrived at empirically based on payment provider SLA data.

Observability must be designed in, not bolted on. Adding distributed tracing, custom business metrics, and structured logging after the fact is significantly more effort than designing observability hooks into every service method from the start. The Micrometer API makes this relatively low-friction — there is no excuse for blind spots.

Appendix A – Project Package Structure

The following is the complete source package hierarchy of the BookMyShow Backend project. Each bounded context is isolated in its own top-level package under com.bookmyshow.

```
com.bookmyshow/
├── BookMyShowApplication.java          # Spring Boot entry point
│
├── user/
│   ├── controller/  UserController.java, AuthController.java
│   ├── service/     UserService.java
│   ├── repository/  UserRepository.java
│   ├── entity/      User.java
│   └── dto/         RegisterRequest, LoginRequest, UserResponse…
│
├── catalog/
│   ├── controller/  MovieController, VenueController, ScreenController…
│   ├── service/     MovieService, CityService, VenueService…
│   ├── repository/  MovieRepository, VenueRepository…
│   ├── entity/      Movie, Genre, City, Venue, Screen, Seat…
│   └── dto/
│
├── show/
│   ├── controller/  ShowController.java
│   ├── service/     ShowService.java
│   ├── repository/  ShowRepository.java
│   └── entity/      Show.java
│
├── booking/
│   ├── controller/  BookingController.java
│   ├── service/     BookingService.java, BookingScheduler.java
│   ├── repository/  BookingRepository, SeatInventoryRepository…
│   └── entity/      Booking, BookingSeat, SeatInventory…
│
├── payment/
│   ├── controller/  PaymentController.java
│   ├── service/     PaymentService.java
│   ├── provider/    RazorpayPaymentProvider.java
│   └── entity/      Payment.java
│
├── notification/
│   ├── service/     NotificationService.java
│   ├── provider/    EmailProvider, SmsProvider, PushProvider…
│   └── entity/      Notification.java
│
```

```
└── common/
    ├── config/      ResilienceConfig, CacheConfig, KafkaConfig…
    ├── security/    SecurityConfig, JwtAuthenticationFilter…
    ├── kafka/       OutboxEventPublisher.java
    ├── exception/   GlobalExceptionHandler, ErrorCode,
BusinessRuleException…
    └── model/       OutboxEvent, IdempotencyKey, ErrorResponse…
```

Figure A.1 — Complete source package hierarchy

Appendix B – Additional Database Table Definitions

B.1 shows Table

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT uuid_generate_v4()	Show identifier
movie_id	UUID	FK → movies(id), NOT NULL	Movie being shown
screen_id	UUID	FK → screens(id), NOT NULL	Physical screen
start_time	TIMESTAMPTZ	NOT NULL	Show start time
end_time	TIMESTAMPTZ	NOT NULL	Show end time
status	VARCHAR(20)	DEFAULT 'UPCOMING'	UPCOMING/ON_SALE/SOLD_OUT/COMPLETED/CANCELLED
language	VARCHAR(50)	NOT NULL	Dubbed/original language
format	VARCHAR(20)	NOT NULL	2D / 3D / IMAX / 4DX
base_price	DECIMAL(10,2)	NOT NULL	Base ticket price for this show
created_at	TIMESTAMPTZ	DEFAULT now()	Record creation timestamp

Table B.01 — shows Table: Column Definitions

B.2 outbox_events Table

Column	Type	Constraints	Description
id	UUID	PK	Event identifier
event_type	VARCHAR(100)	NOT NULL	e.g. BookingCreated, PaymentSucceeded
aggregate_type	VARCHAR(50)	NOT NULL	e.g. BOOKING, PAYMENT, CATALOG
aggregate_id	VARCHAR(100)	NOT NULL	ID of the source entity
payload	JSONB	NOT NULL	Serialized event payload
topic	VARCHAR(100)	NOT NULL	Target Kafka topic
status	VARCHAR(20)	DEFAULT 'PENDING'	PENDING / PUBLISHED / FAILED
retry_count	INTEGER	DEFAULT 0	Number of publish attempts
created_at	TIMESTAMPTZ	DEFAULT now()	Event creation time
published_at	TIMESTAMPTZ	NULLABLE	Kafka ACK timestamp

Table B.02 — outbox_events Table: Column Definitions

B.3 payments Table

Column	Type	Constraints	Description
id	UUID	PK	Payment record ID
booking_id	UUID	FK → bookings(id), UNIQUE	Owning booking (1:1)
amount	DECIMAL(10,2)	NOT NULL	Payment amount
currency	VARCHAR(10)	DEFAULT 'INR'	ISO 4217 currency code
status	VARCHAR(30)	DEFAULT 'INITIATED'	Payment lifecycle state
provider	VARCHAR(50)	NOT NULL	RAZORPAY / STRIPE / etc.
provider_order_id	VARCHAR(200)	NULLABLE	Provider order reference
provider_payment_id	VARCHAR(200)	NULLABLE, UNIQUE	Provider payment reference
idempotency_key	VARCHAR(100)	UNIQUE	Client dedup key
failure_reason	TEXT	NULLABLE	Provider failure message
refund_amount	DECIMAL(10,2)	NULLABLE	Refunded amount (partial/full)
created_at	TIMESTAMPTZ	DEFAULT now()	Record creation time

Table B.03 — payments Table: Column Definitions

B.4 Key Database Indexes

The following indexes are defined across the schema to support common query patterns and ensure optimal performance for high-frequency lookups:

Table	Index Name	Columns	Purpose
users	idx_users_email	email	Login lookup by email
users	idx_users_phone	phone	Login lookup by phone
seat_inventory	idx_seat_inv_show_status	show_id, status	Available seat queries
seat_inventory	idx_seat_inv_hold_expires	hold_expires_at, status	Expired hold cleanup
bookings	idx_bookings_user_id	user_id	User booking history
bookings	idx_bookings_idempotency	idempotency_key	Idempotency checks
outbox_events	idx_outbox_status_created	status, created_at	Unpublished event polling
payments	idx_payments_provider_pid	provider_payment_id	Webhook lookup
shows	idx_shows_movie_screen	movie_id, screen_id, start_time	Show schedule queries

Table B.04 — Key Database Indexes

References

[1] Spring Framework Team. Spring Boot Reference Documentation, Version 3.2.5. Pivotal/VMware, 2024. https://docs.spring.io/spring-boot/docs/3.2.5/reference/

[2] Spring Security Team. Spring Security Reference — OAuth2 Resource Server and JWT. Pivotal/VMware, 2024. https://docs.spring.io/spring-security/reference/

[3] Kleppmann, M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, 2017. ISBN: 978-1-4920-3205-1.

[4] Richardson, C. Microservices Patterns: With Examples in Java. Manning Publications, 2018. ISBN: 978-1-61729-454-9.

[5] Resilience4j Authors. Resilience4j User Guide, Version 2.2.0. 2023. https://resilience4j.readme.io/

[6] PostgreSQL Global Development Group. PostgreSQL 16 Documentation — Chapter 13: Concurrency Control and Explicit Locking. 2024. https://www.postgresql.org/docs/16/explicit-locking.html

[7] Apache Kafka Project. Kafka Documentation — Producer Idempotency and Transactions. Apache Software Foundation, 2024. https://kafka.apache.org/documentation/

[8] Flyway by Redgate. Flyway Documentation, Version 10.x. 2024. https://documentation.red-gate.com/fd/

[9] Micrometer Project. Micrometer Observability Reference Documentation. 2024. https://micrometer.io/docs

[10] Testcontainers Authors. Testcontainers for Java — Getting Started. 2024. https://java.testcontainers.org/

[11] Keycloak Project. Keycloak Server Administration Guide, Version 24.0. Red Hat, 2024. https://www.keycloak.org/docs/24.0/

[12] Kubernetes Authors. Kubernetes Documentation — Horizontal Pod Autoscaling. CNCF, 2024. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[13] HikariCP Authors. HikariCP — Fast, Simple, Reliable. Connection Pool for JDBC. 2024. https://github.com/brettwooldridge/HikariCP

[14] Lettuce Authors. Lettuce — Advanced Java Redis client. 2024. https://lettuce.io/

[15] SpringDoc OpenAPI Authors. SpringDoc OpenAPI Library Documentation. 2024. https://springdoc.org/

[16] MapStruct Authors. MapStruct Reference Guide, Version 1.5.5. 2024. https://mapstruct.org/documentation/stable/reference/html/

[17] Razorpay. Razorpay Payment Gateway — Developer Documentation. 2024. https://razorpay.com/docs/

[18] Poutanen, T. et al. Identifying and Mitigating Database Connection Pool Exhaustion in Microservice Deployments. In: Proceedings of the IEEE International Conference on Cloud Engineering, 2023.

[19] Newman, S. Building Microservices: Designing Fine-Grained Systems. 2nd ed. O'Reilly Media, 2021. ISBN: 978-1-4920-3434-5.

[20] Nygard, M.T. Release It! Design and Deploy Production-Ready Software. 2nd ed. Pragmatic Bookshelf, 2018. ISBN: 978-1-68050-239-8.